

Experiment 7 – TD(0)

NAME:N.AKSHAYA

REG.NO:192425129

COURSE CODE:MLA0305

COURSE NAME:REINFORCEMENT LEARNING

SOLT:B

7.Temporal Difference Learning using TD(0) Algorithm.

Aim:

To implement Temporal Difference Learning using the TD(0) algorithm in Python and estimate the value function of states by updating state values from experience without waiting until the end of an episode. The experiment also provides a summary and visual representation of the learned state values.

Procedure:

1. Define the states and initialize their value estimates.
2. Set the learning rate α and discount factor γ .
3. Generate episodes containing states and rewards.
4. Start with an initial state.
5. Observe the next state and the immediate reward.
6. Calculate the TD Target using:

$$\text{TD Target} = R + \gamma V(S')$$

7. Calculate the TD Error using:

$$\text{TD Error} = [R + \gamma V(S')] - V(S)$$

8. Update the current state's value using:

$$V(S) = V(S) + \alpha[\text{TD Error}]$$

9. Repeat the process for multiple episodes.
10. Display the final state values.
11. Plot a graph showing the learned value of each state.
12. Display a summary of the TD(0) learning process.

Python code:

```
# TEMPORAL DIFFERENCE LEARNING USING TD(0) ALGORITHM
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
# 1. DEFINE STATES
```

```
states = ["S0", "S1", "S2", "S3", "Goal"]
```

```
# Initial value of each state
```

```
V = {
```

```
    "S0": 0.0,
```

```
    "S1": 0.0,
```

```
    "S2": 0.0,
```

```
    "S3": 0.0,
```

```
    "Goal": 0.0
```

```
}
```

```
# 2. PARAMETERS
```

```
alpha = 0.1    # Learning rate
```

```
gamma = 0.9    # Discount factor
```

```
episodes = 100 # Number of episodes
```

3. ENVIRONMENT

State, Next State, Reward

```
experience = [  
    ("S0", "S1", 1),  
    ("S1", "S2", 2),  
    ("S2", "S3", 3),  
    ("S3", "Goal", 10)  
]
```

Store state values after every episode

```
history = []
```

4. TD(0) LEARNING

for episode in range(epochs):

for state, next_state, reward in experience:

TD Target

td_target = reward + gamma * V[next_state]

TD Error

td_error = td_target - V[state]

TD(0) Value Update

V[state] = V[state] + alpha * td_error

Store current values

```
history.append([
```

V["S0"],

V["S1"],

V["S2"],

```

    V["S3"],
    V["Goal"]

])

# Convert history into NumPy array
history = np.array(history)

# 5. DISPLAY RESULTS

print("\n" + "=" * 60)

print("    TEMPORAL DIFFERENCE LEARNING USING TD(0)")

print("=" * 60)

print("\nLearning Parameters")

print("-" * 40)

print("Learning Rate (alpha) :", alpha)

print("Discount Factor (gamma):", gamma)

print("Number of Episodes   :", episodes)

print("\nFinal State Values")

print("-" * 40)

for state in states:

    print(f"{state:8s} : {V[state]:.4f}")

# 6. SUMMARY

print("\n" + "=" * 60)

print("SUMMARY")

print("=" * 60)

print("TD(0) learns the value of each state from experience.")

print("The TD target is calculated using:")

```

```
print("TD Target = Reward + gamma * V(Next State)")

print("\nThe TD error is calculated using:")

print("TD Error = TD Target - V(Current State)")

print("\nThe value is updated using:")

print("V(State) = V(State) + alpha * TD Error")

print("\nLearning completed successfully.")
```

7. DIFFERENT COLORS

```
colors = [

    "red",

    "blue",

    "green",

    "orange",

    "purple"

]
```

8. GRAPH 1 - FINAL STATE VALUES

```
plt.figure(figsize=(9, 6))

final_values = [V[state] for state in states]

bars = plt.bar(

    states,

    final_values,

    color=colors,

    edgecolor="black"

)
```

```
plt.title(  
    "TD(0) Learned State Values",  
    fontsize=16,  
    fontweight="bold"  
)  
  
plt.xlabel(  
    "States",  
    fontsize=12  
)  
  
plt.ylabel(  
    "Learned Value",  
    fontsize=12  
)  
  
# Display value above each bar  
for bar, value in zip(bars, final_values):  
    plt.text(  
        bar.get_x() + bar.get_width() / 2,  
        bar.get_height() + 0.2,  
        f"{value:.2f}",  
        ha="center",  
        va="bottom",  
        fontsize=11,  
        fontweight="bold"  
    )
```

```
plt.grid(
    axis="y",
    linestyle="--",
    alpha=0.5
)

plt.tight_layout()

plt.show()

# 9. GRAPH 2 - TD(0) LEARNING CURVE

plt.figure(figsize=(10, 6))

for i in range(len(states)):

    plt.plot(
        range(1, episodes + 1),
        history[:, i],
        color=colors[i],
        linewidth=2,
        label=states[i]
    )

plt.title(
    "TD(0) Learning Curve",
    fontsize=16,
    fontweight="bold"
)

plt.xlabel(
    "Episode",
```

```
        fontsize=12
    )
    plt.ylabel(
        "State Value",
        fontsize=12
    )
    plt.legend(
        title="States"
    )
    plt.grid(
        True,
        linestyle="--",
        alpha=0.5
    )
    plt.tight_layout()
    plt.show()
```

Result:

Thus, Temporal Difference Learning using the TD(0) algorithm was successfully implemented in Python. The value function of the states was estimated using the TD target and TD error, and the learning process was visualized using state-value and learning-curve graphs.

Summary Result:

```
=====
      TEMPORAL DIFFERENCE LEARNING USING TD(0)
=====
```

Learning Parameters

```
-----
Learning Rate (alpha) : 0.1
Discount Factor (gamma): 0.9
Number of Episodes    : 100
```

Final State Values

```
-----
S0      : 12.4575
S1      : 12.7833
S2      : 11.9970
S3      : 9.9997
Goal    : 0.0000
```

SUMMARY

```
=====
TD(0) learns the value of each state from experience.
The TD target is calculated using:
TD Target = Reward + gamma * V(Next State)
```

The TD error is calculated using:
TD Error = TD Target - V(Current State)

The value is updated using:
 $V(\text{State}) = V(\text{State}) + \alpha * \text{TD Error}$

Learning completed successfully.

Visualization Result:

Learning Completed Successfully

